Chapter 16

Security at the Application Layer: PGP and S/MIME

Chapter 16

Objectives

- To explain the general structure of an e-mail application program
- To discuss how PGP can provide security services for e-mail
- To discuss how S/MIME can provide security services for e-mail
- To define trust mechanism in both PGP and S/MIME
- To show the structure of messages exchanged in PGP and S/MIME

16-1 E-MAIL

Let us first discuss the electronic mail (e-mail) system in general.

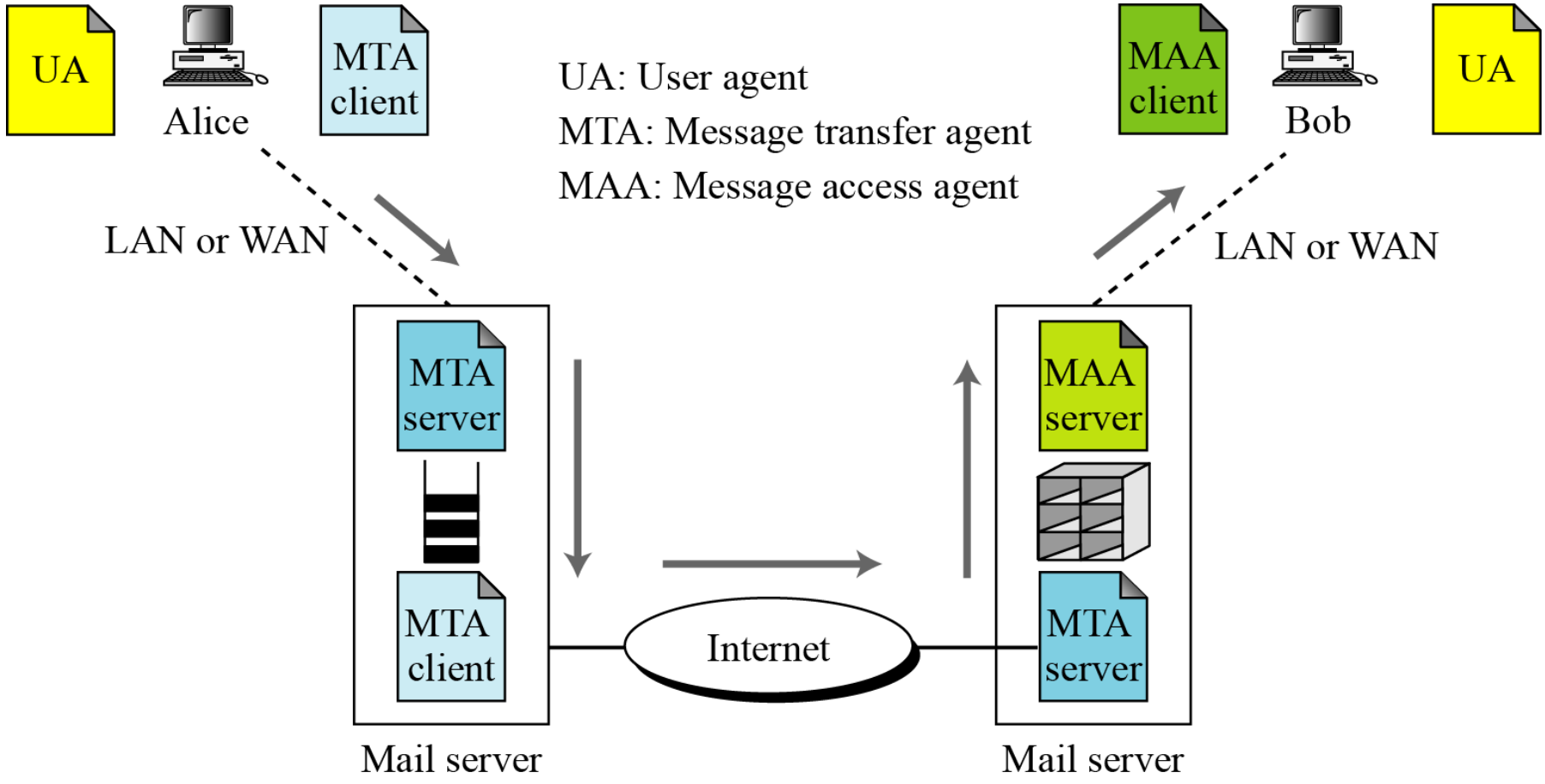
Topics discussed in this section:

16.1.1 E-mail Architecture

16.1.2 E-mail Security

16.1.1 E-mail Architecture

Figure 16.1 *E-mail architecture*



- is a store and retrieve activity/ push-pull program.

16.1.2 E-mail Security

A sends the message to B's mailbox.
B reads it, sometimes later and may or may not reply.
So A and B does not create a session.

Cryptographic Algorithms

Note

In e-mail security, the sender of the message needs to include the name or identifiers of the algorithms used in the message. (like: 3DES/MD5)

Cryptographic Secrets

Note

In e-mail security, the encryption/decryption is done using a symmetric-key algorithm, but the secret key to decrypt the message is encrypted with the public key of the receiver and is sent with the message.

Certificates

It is obvious that some public-key algorithms must be used for e-mail security.

To encrypt the secret key, A needs B's public key.

To verify a signed message, B needs A's public key.

16-2 PGP

Pretty Good Privacy (PGP) can be used to create a secure e-mail message or to store a file securely for future retrieval.

Topics discussed in this section:

16.2.1 Scenarios
16.2.2 Key Rings
16.2.3 PGP Certificates
16.2.4 Key Revocation
16.2.5 Extracting Information from Rings
16.2.6 PGP Packets
16.2.7 PGP Messages

PGP

It was originally created by Philip Zimmermann in 1991.

Services:

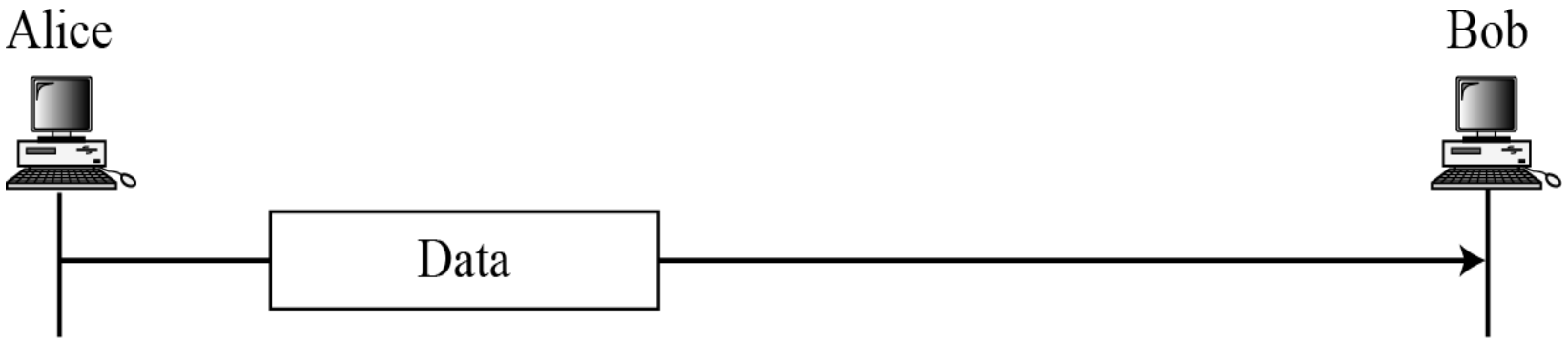
PGP provides for five services:

1.Authentication
2.Confidentiality
3.Compression
4.E-mail compatibility
5.segmentation

16.2.1 Scenarios

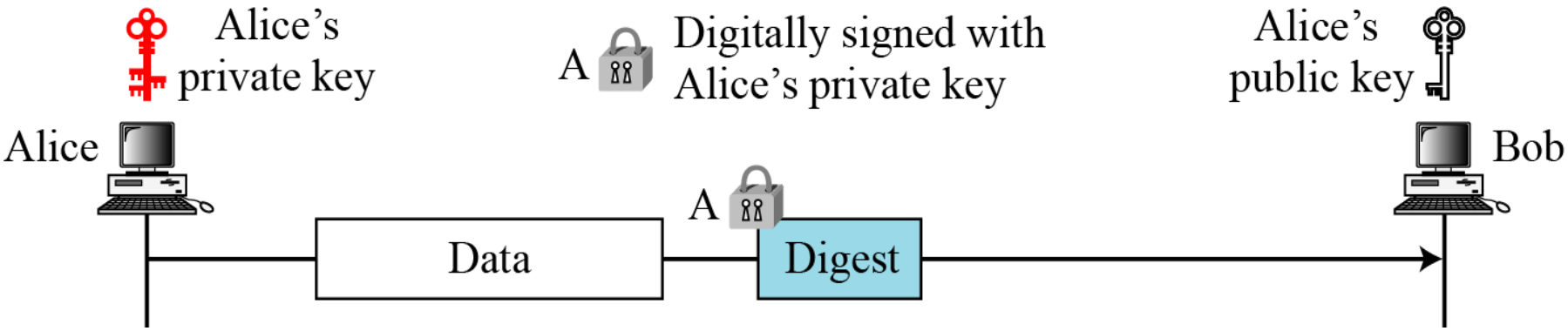
Plaintext

Figure 16.2 *A plaintext message*



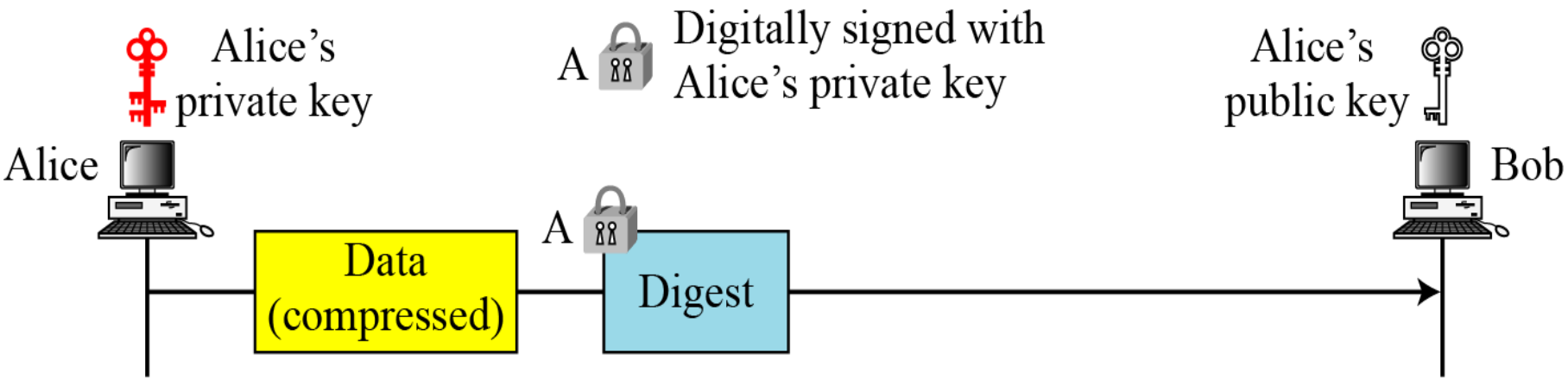
Message Integrity

Figure 16.3 *An authenticated message*



Compression

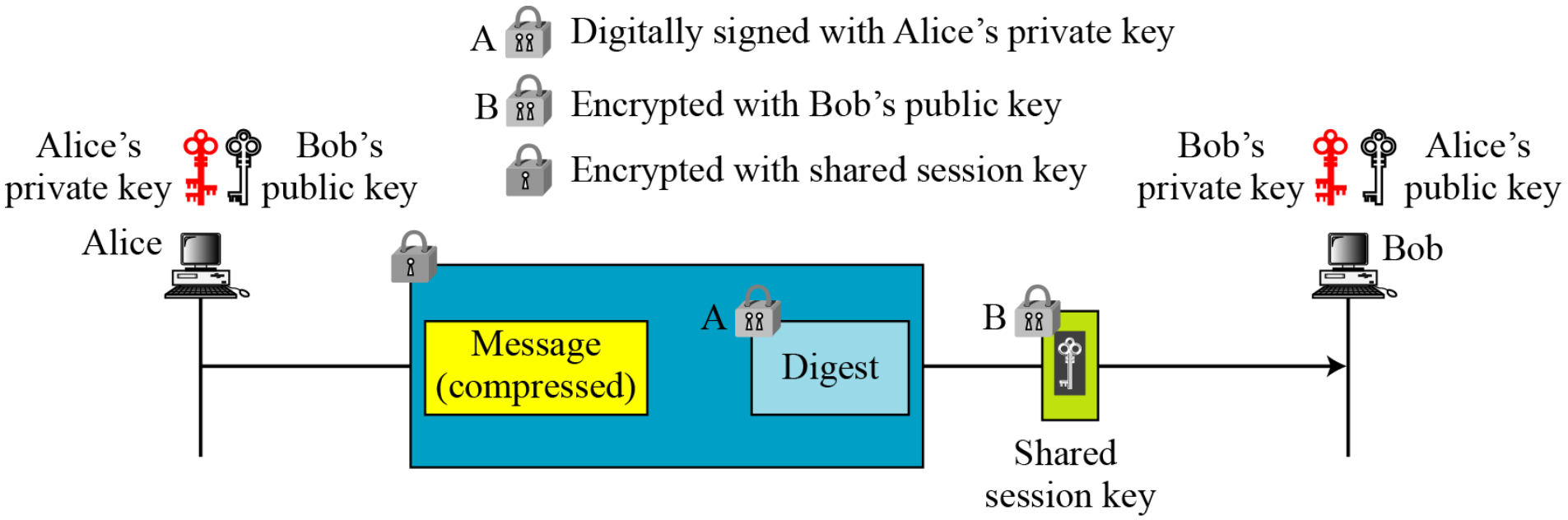
Figure 16.4 *A compressed message*



16.2.1 Continued

Confidentiality with One-Time Session Key

Figure 16.5 *A confidential message*



16.2.1 Continued

Code Conversion

Another service provided by PGP is code conversion. Only ASCII characters are allowed in most e-mail systems. Each character to be sent (after encryption) is converted to Radix-64 code.

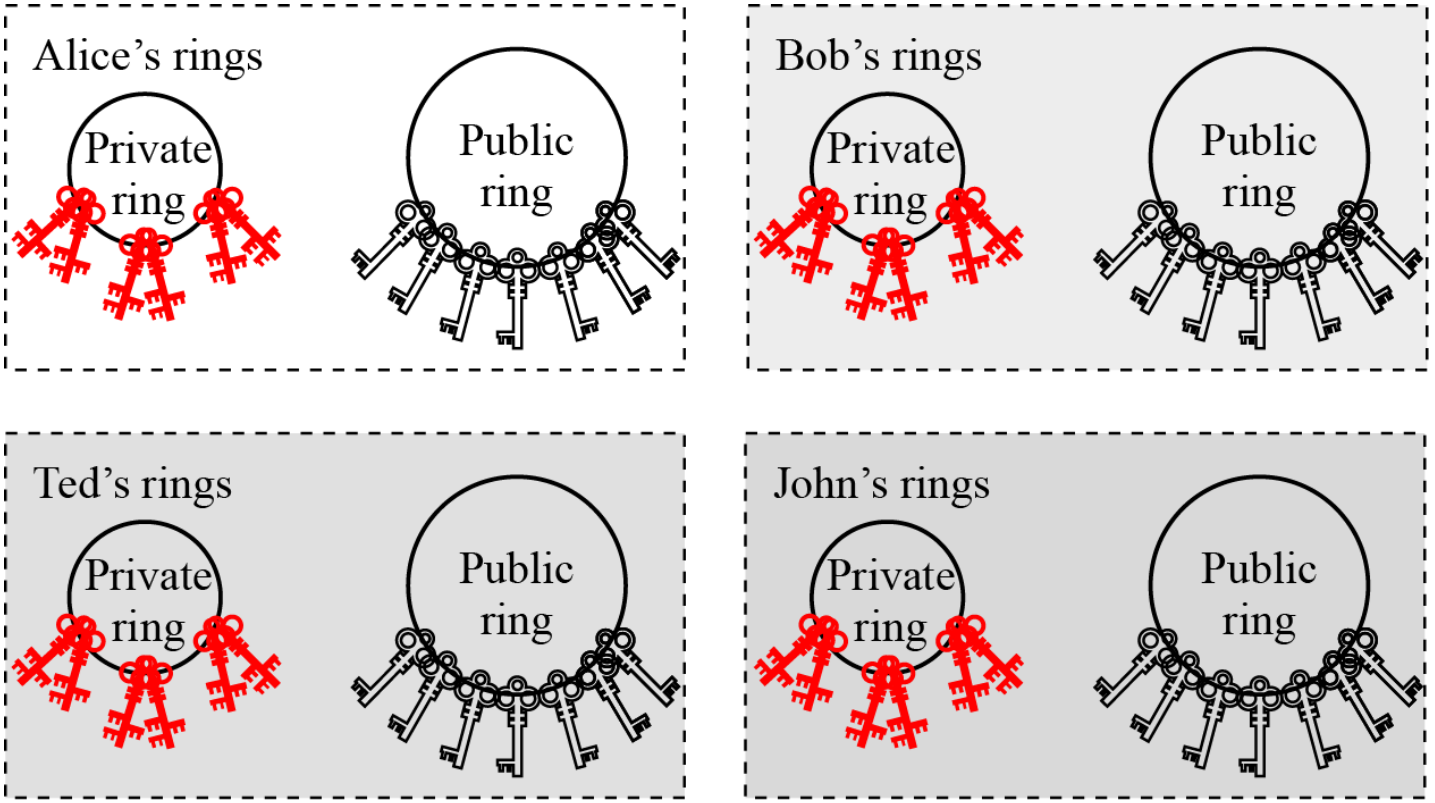
Segmentation

PGP allows segmentation of the message.

16.2.2 Key Rings

Figure 16.6 *Key rings in PGP*

***A** may need to send messages to many people.*



16.2.2 Continued

PGP Algorithms

Table 16.1 *Public-key algorithms*

ID	*Description*
1	RSA (encryption or signing)
2	RSA (for encryption only)
3	RSA (for signing only)
16	ElGamal (encryption only)
17	DSS
18	Reserved for elliptic curve
19	Reserved for ECDSA
20	ElGamal (for encryption or signing)
21	Reserved for Diffie-Hellman
100–110	Private algorithms

16.2.2 Continued

Table 16.2 *Symmetric-key algorithms*

ID	*Description*
0	No Encryption
1	IDEA
2	Triple DES
3	CAST-128
4	Blowfish
5	SAFER-SK128
6	Reserved for DES/SK
7	Reserved for AES-128
8	Reserved for AES-192
9	Reserved for AES-256
100–110	Private algorithms

Table 16.3 *Hash Algorithms*

ID	*Description*
1	MD5
2	SHA-1
3	RIPE-MD/160
4	Reserved for double-width SHA
5	MD2
6	TIGER/192
7	Reserved for HAVAL
100–110	Private algorithms

Table 16.4 *Compression methods*

ID	*Description*
0	Uncompressed
1	ZIP
2	ZLIP
100–110	Private methods

16.2.3 PGP Certificates

X.509 Certificates

Protocols that use X.509 certificates depend on the hierarchical structure of the trust.

Note

In X.509, there is a single path from the fully trusted authority to any certificate.

PGP Certificates

In PGP, there is no need for CAs; anyone in the ring can sign a certificate for anyone else in the ring. There is no tree/hierarchy of trust.

Note

In PGP, there can be multiple paths from fully or partially trusted authorities to any subject.

Trusts and Legitimacy

The entire operation of PGP is based on introducer trust, the certificate trust, and the legitimacy of the public keys.

Figure 16.7 *Format of private key ring table*



User ID	Key ID	Public key	Encrypted private key	Timestamp
⋮	⋮	⋮	⋮	⋮

16.2.3 Continued

Example 16.1

Let us show a private key ring table for Alice. We assume that Alice has only two user IDs, alice@some.com and alice@anet.net. We also assume that Alice has two sets of private/public keys, one for each user ID.

Table 16.5 *Private key ring table for Example 1*

User ID	*Key ID*	*Public Key*	*Encrypted Private Key*	*Timestamp*
alice@anet.net	AB13...45	AB13...45...59	**32452398...23**	031505-16:23
alice@some.com	FA23...12	FA23...12...22	**564A4923...23**	031504-08:11

Figure 16.8 *Format of a public key ring table*



Public ring

User ID	Key ID	Public key	Producer trust	Certificate(s)	Certificate trust(s)	Key Legitimacy	Timestamp
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

16.2.3 Continued

Example 16.2

A series of steps will show how a public key ring table is formed for Alice.

Table 16.6 *Example 2, starting table*

User ID	*Key ID*	*Public key*	*Prod. trust*	*Certificate*	*Cert. trust*	*Key legit.*	*Time-stamp*
Alice...	AB...	AB.....	F			F	.....

Table 16.7 *Example 2, after Bob is added to the table*

User ID	*Key ID*	*Public key*	*Prod. trust*	*Certificate*	*Cert. trust*	*Key legit.*	*Time-stamp*
Alice...	AB...	AB.....	F			F	.....
Bob...	12...	12.....	F			F	.....

Example 16.2 Continued

Table 16.8 *Example 2, after Ted is added to the table*

User ID	*Key ID*	*Public key*	*Prod. trust*	*Certificate*	*Cert. trust*	*Key legit.*	*Time-stamp*
Alice...	AB...	AB.....	F			F	.....
Bob...	12...	12.....	F			F	.....
Ted...	48...	48.....	F	Bob's	F	F	.....

Table 16.9 *Example 2, after Anne is added to the table*

User ID	*Key ID*	*Public key*	*Prod. trust*	*Certificate*	*Cert. trust*	*Key legit.*	*Time-stamp*
Alice...	AB...	AB.....	F			F	.....
Bob...	12...	12.....	F			F	.....
Ted...	48...	48.....	F	Bob's	F	F	.....
Anne...	71...	71.....	P	Bob's	F	F	.....

16-3 S/MIME

Another security service designed for electronic mail is Secure/Multipurpose Internet Mail Extension (S/MIME).
The protocol is an enhancement of the Multipurpose Internet Mail Extension (MIME) protocol.

Topics discussed in this section:

16.3.1 MIME
16.3.2 S/MIME
16.3.3 Applications of S/MIME

16.3.1 Continued

SMTP, supports only 7-bit ASCII characters

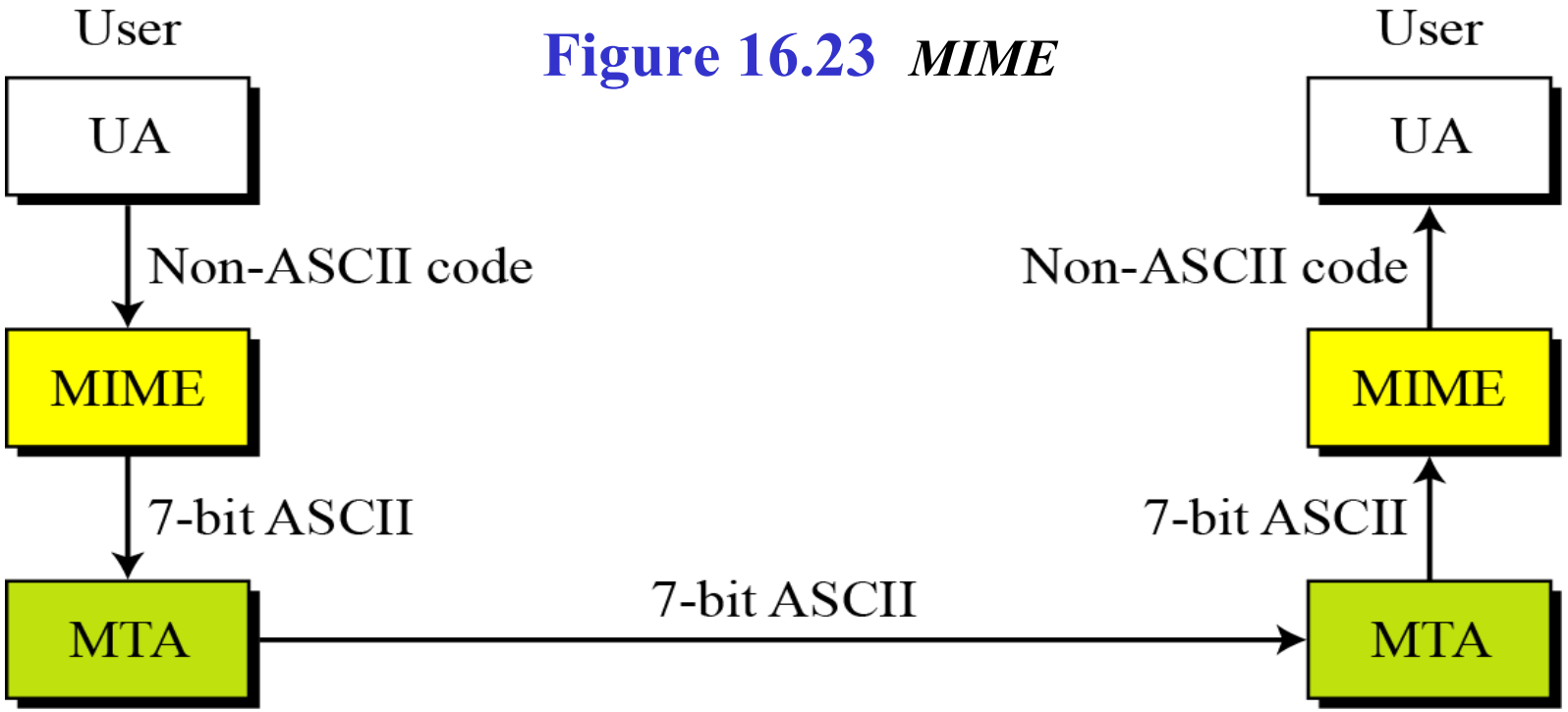


Figure 16.23 ***MIME***

ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

All 128 ASCII characters including non-printable characters

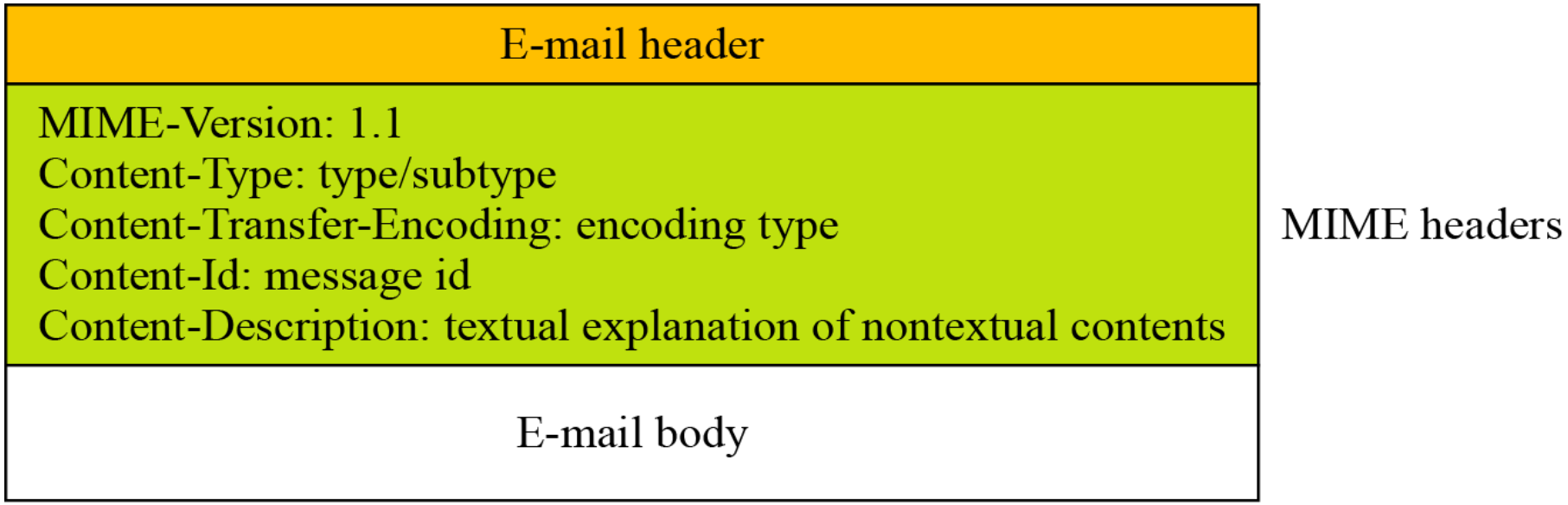
(represented by their abbreviation).

The 95 ASCII graphic characters are numbered from 0x20 to 0x7E

(32 to 126 decimal).

The space character was initially considered to be a non-printing

Figure 16.24 *Teledesic*



16.3.1 Continued

MIME-Version

This header defines the version of MIME used. The current version is 1.1.

MIME-Version: 1.1

Content-Type

The content type and the content subtype are separated by a slash. Depending on the subtype, the header may contain other parameters.

Content-Type: <type / subtype; parameters>

16.3.1 Continued

Table 16.14 *Data types and subtypes in MIME*

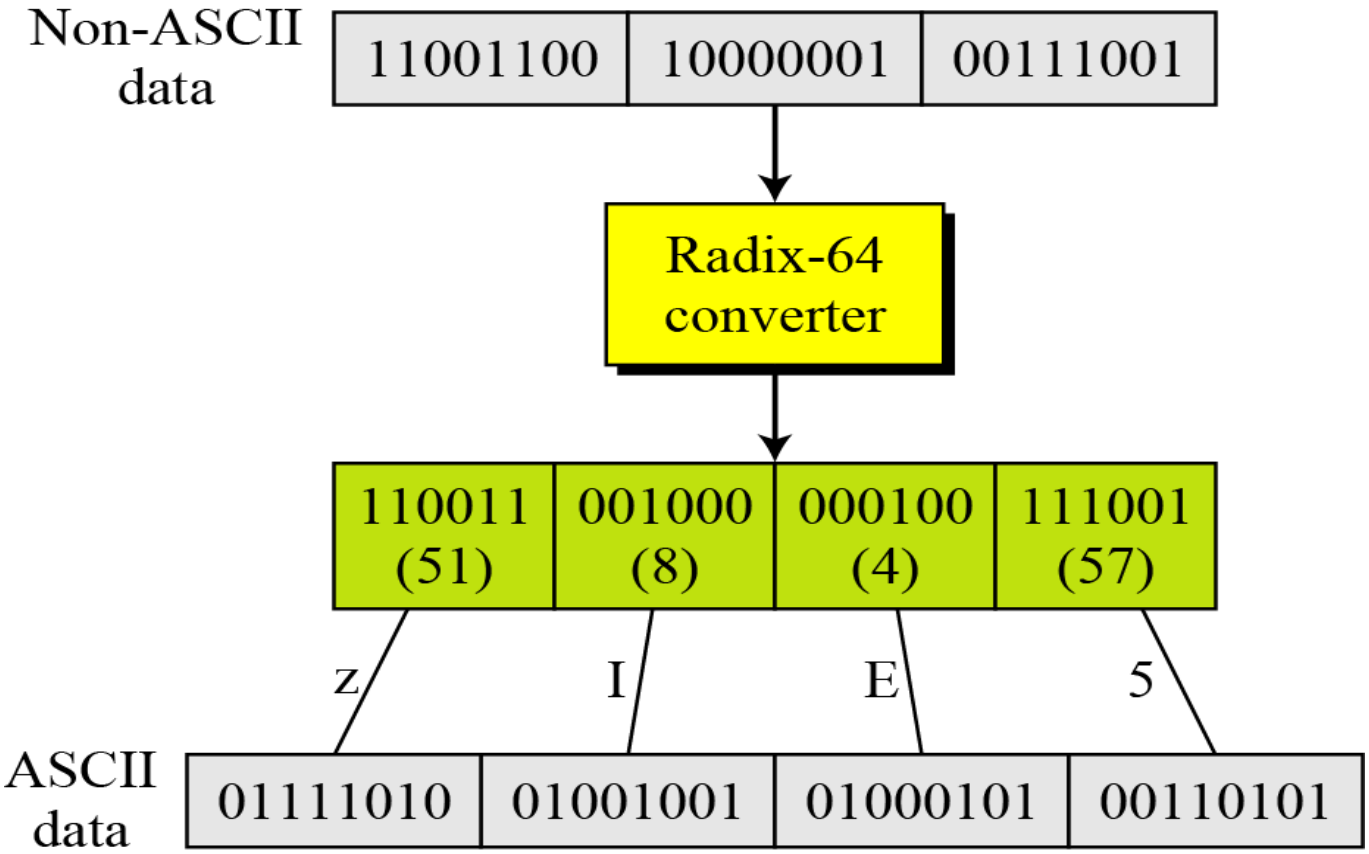
Type	*Subtype*	*Description*
Text	Plain	Unformatted.
	HTML	HTML format.
Multipart	Mixed	Body contains ordered parts of different data types.
	Parallel	Same as above, but no order.
	Digest	Similar to Mixed, but the default is message/RFC822.
	Alternative	Parts are different versions of the same message.
Message	RFC822	Body is an encapsulated message.
	Partial	Body is a fragment of a bigger message.
	External-Body	Body is a reference to another message.
Image	JPEG	Image is in JPEG format.
	GIF	Image is in GIF format.
Video	MPEG	Video is in MPEG format.
Audio	Basic	Single channel encoding of voice at 8 KHz.
Application	PostScript	Adobe PostScript.
	Octet-stream	General binary data (eight-bit bytes).

16.3.1 Continued

Table 16.15 *Content-transfer-encoding*

Type	*Description*
7bit	NVT ASCII characters and short lines.
8bit	Non-ASCII characters and short lines.
Binary	Non-ASCII characters with unlimited-length lines.
Radix-64	6-bit blocks of data are encoded into 8-bit ASCII characters using Radix-64 conversion.
Quoted-printable	Non-ASCII characters are encoded as an equal sign followed by an ASCII code.

Figure 16.25 *Radix-64 conversion*

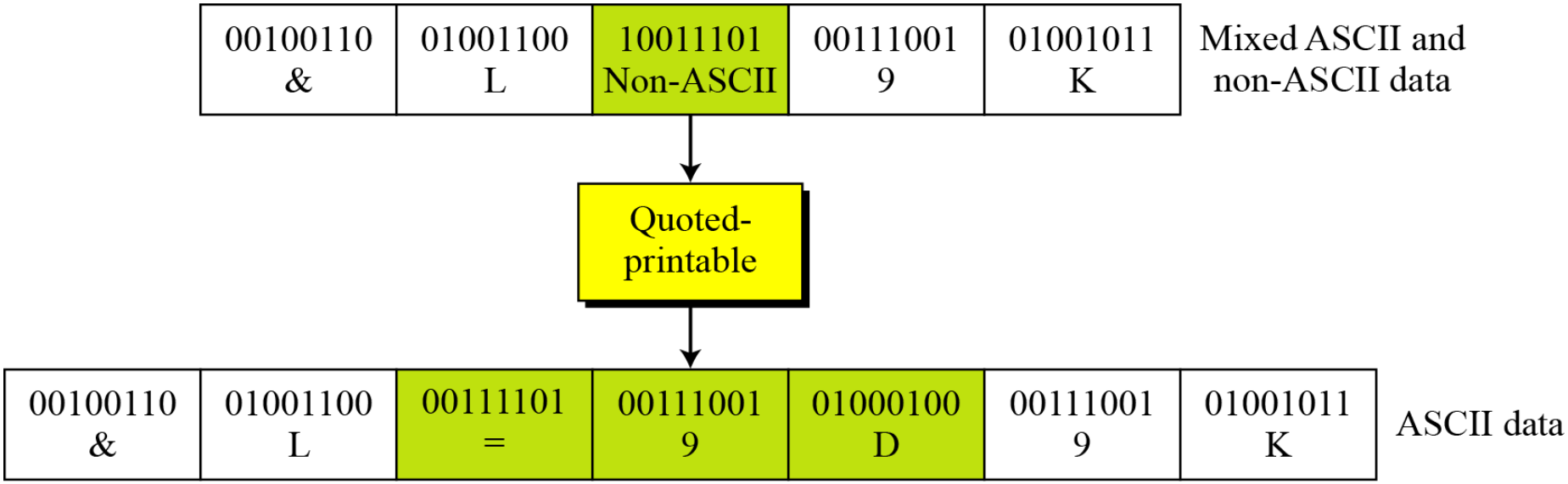


16.3.1 Continued

Table 16.16 *Radix-64 encoding table*

Value	Code	Value	Code	Value	Code	Value	Code	Value	Code	Value	Code
0	**A**	11	**L**	22	**W**	33	**h**	44	**s**	55	**3**
1	**B**	12	**M**	23	**X**	34	**i**	45	**t**	56	**4**
2	**C**	13	**N**	24	**Y**	35	**j**	46	**u**	57	**5**
3	**D**	14	**O**	25	**Z**	36	**k**	47	**v**	58	**6**
4	**E**	15	**P**	26	**a**	37	**l**	48	**w**	59	**7**
5	**F**	16	**Q**	27	**b**	38	**m**	49	**x**	60	**8**
6	**G**	17	**R**	28	**c**	39	**n**	50	**y**	61	**9**
7	**H**	18	**S**	29	**d**	40	**o**	51	**z**	62	+
8	**I**	19	**T**	30	**e**	41	**p**	52	**0**	63	/
9	**J**	20	**U**	31	**f**	42	**q**	53	**1**		
10	**K**	21	**V**	32	**g**	43	**r**	54	**2**		

Figure 16.26 *Quoted-printable*



16.3.2 S/MIME

S/MIME adds some new content types to include security services to the MIME. All of these new types include the parameter "application/pkcs7-mime,"

"pkcs" - "Public Key Cryptography Specification."

Cryptographic Message Syntax (CMS)

To define how security services, such as confidentiality or integrity, can be added to MIME content types, S/MIME has defined Cryptographic Message Syntax (CMS).

The syntax in each case defines the exact encoding scheme for each content type.

For details, refer to RFC 3369 and 3370.

Figure 16.27 *Signed-data content type*

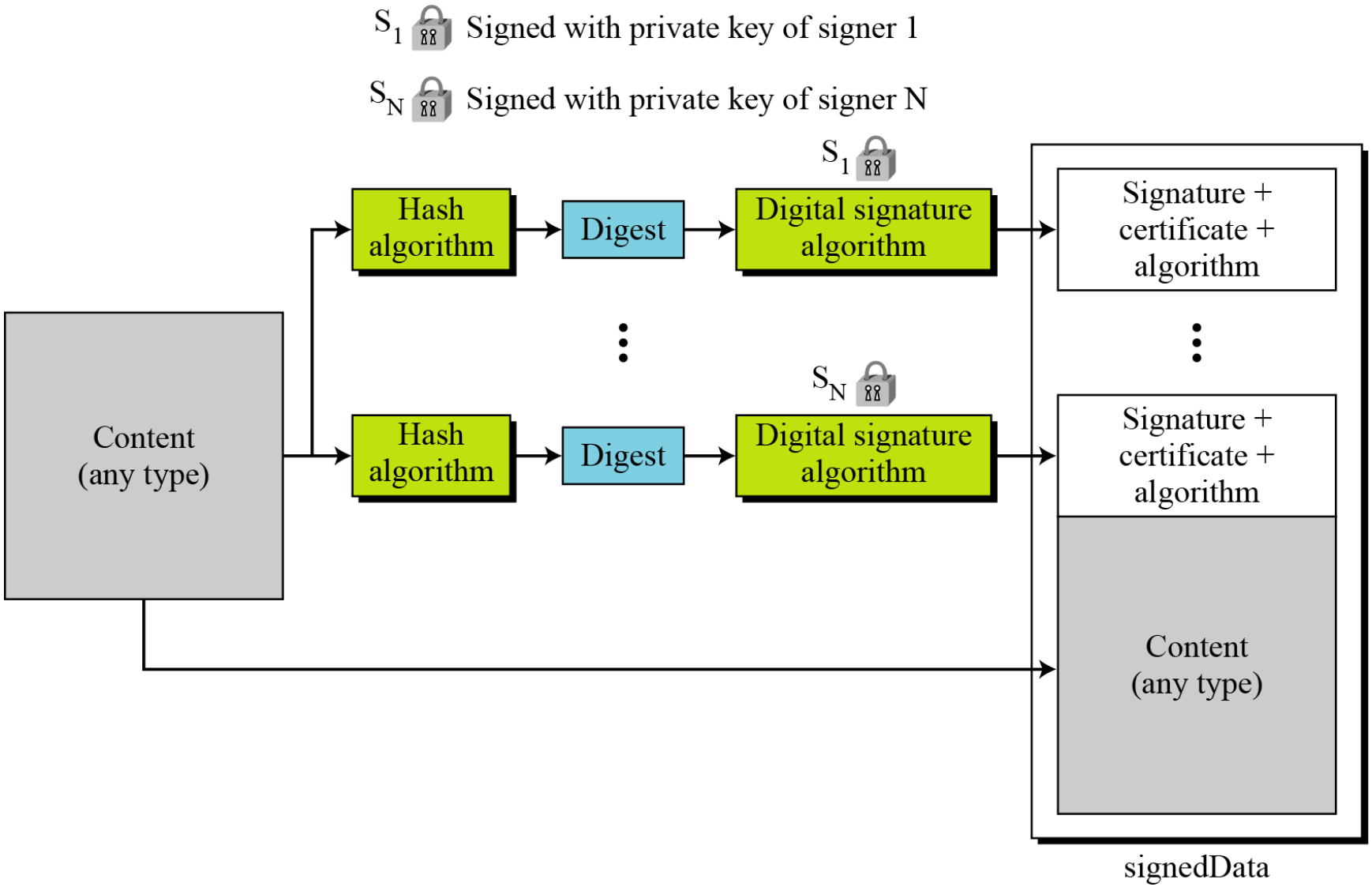


Figure 16.28 *Enveloped-data content type*

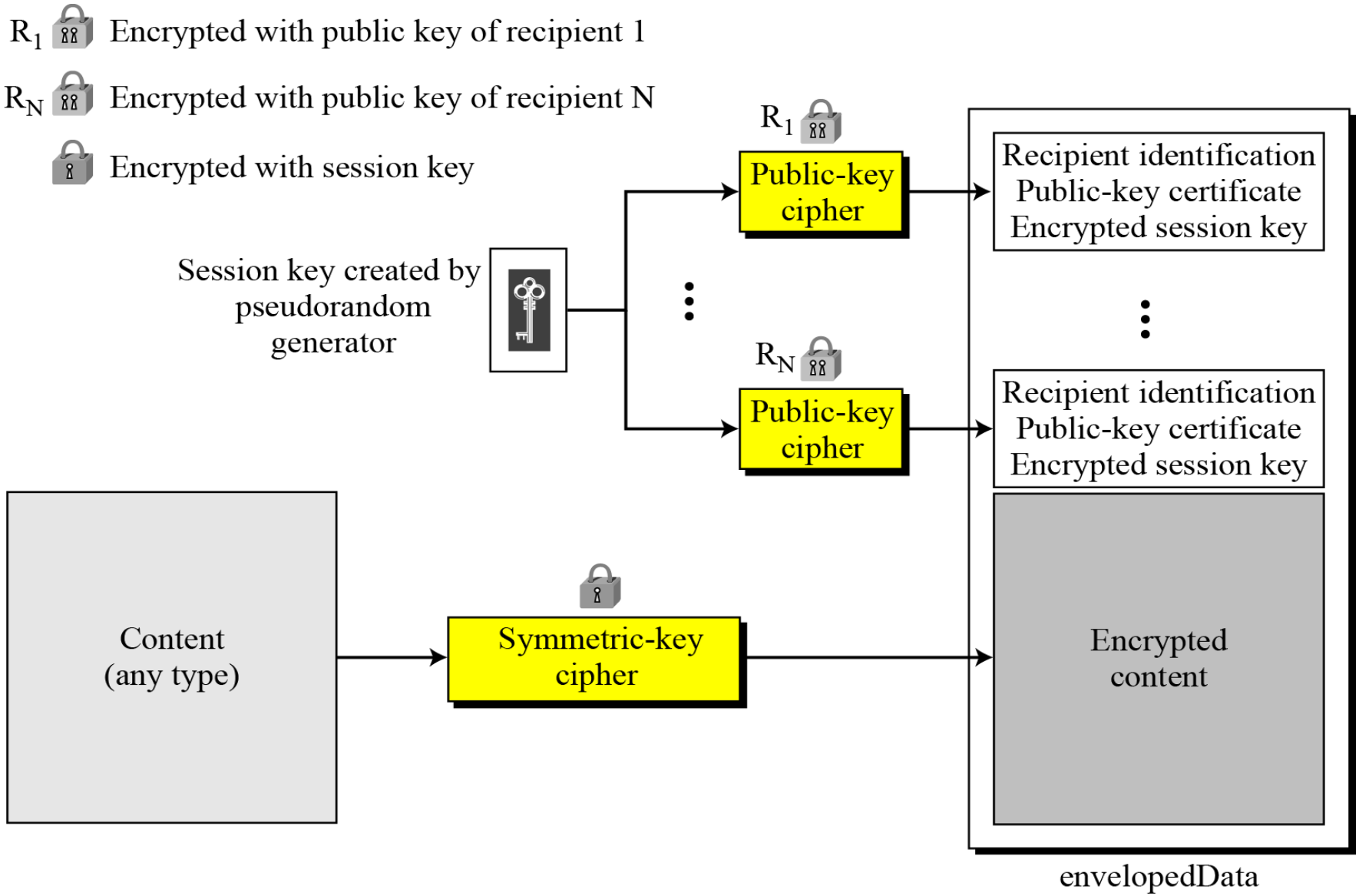


Figure 16.29 *Digest-data content type*

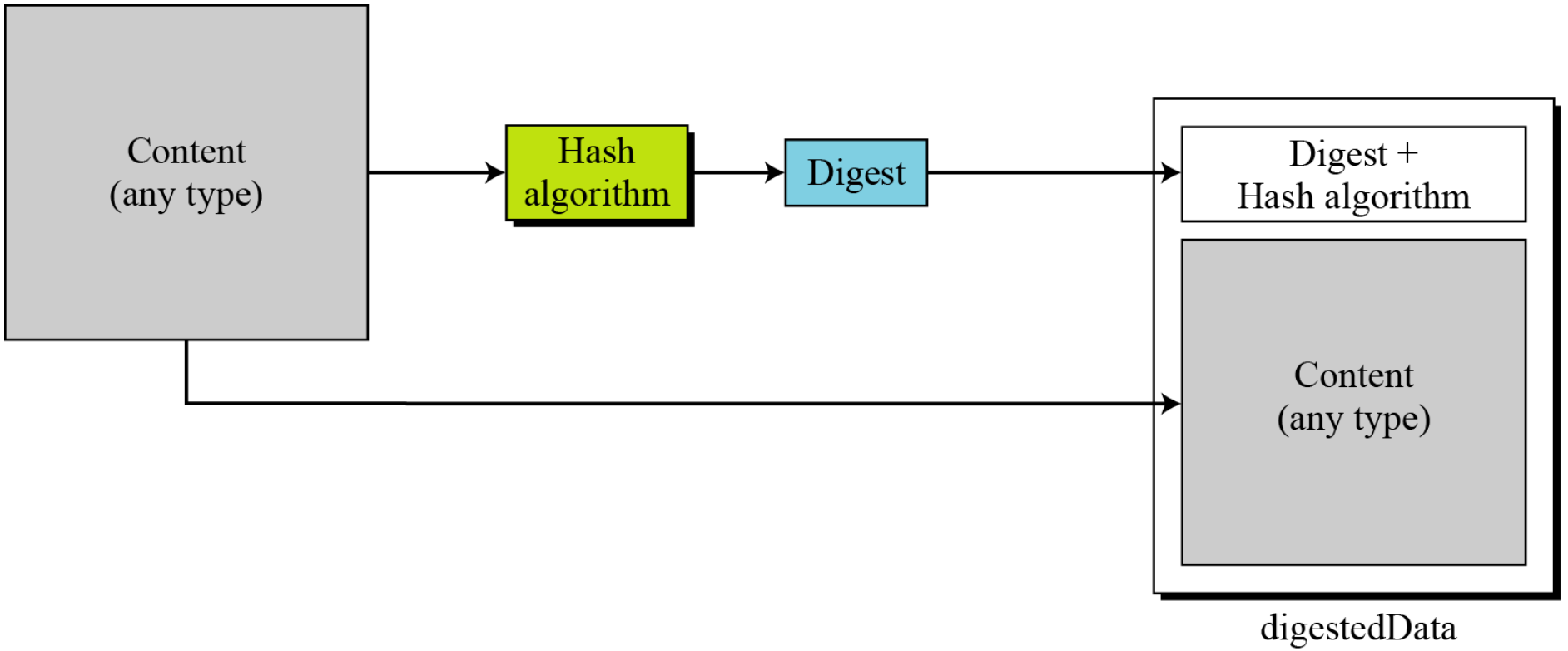
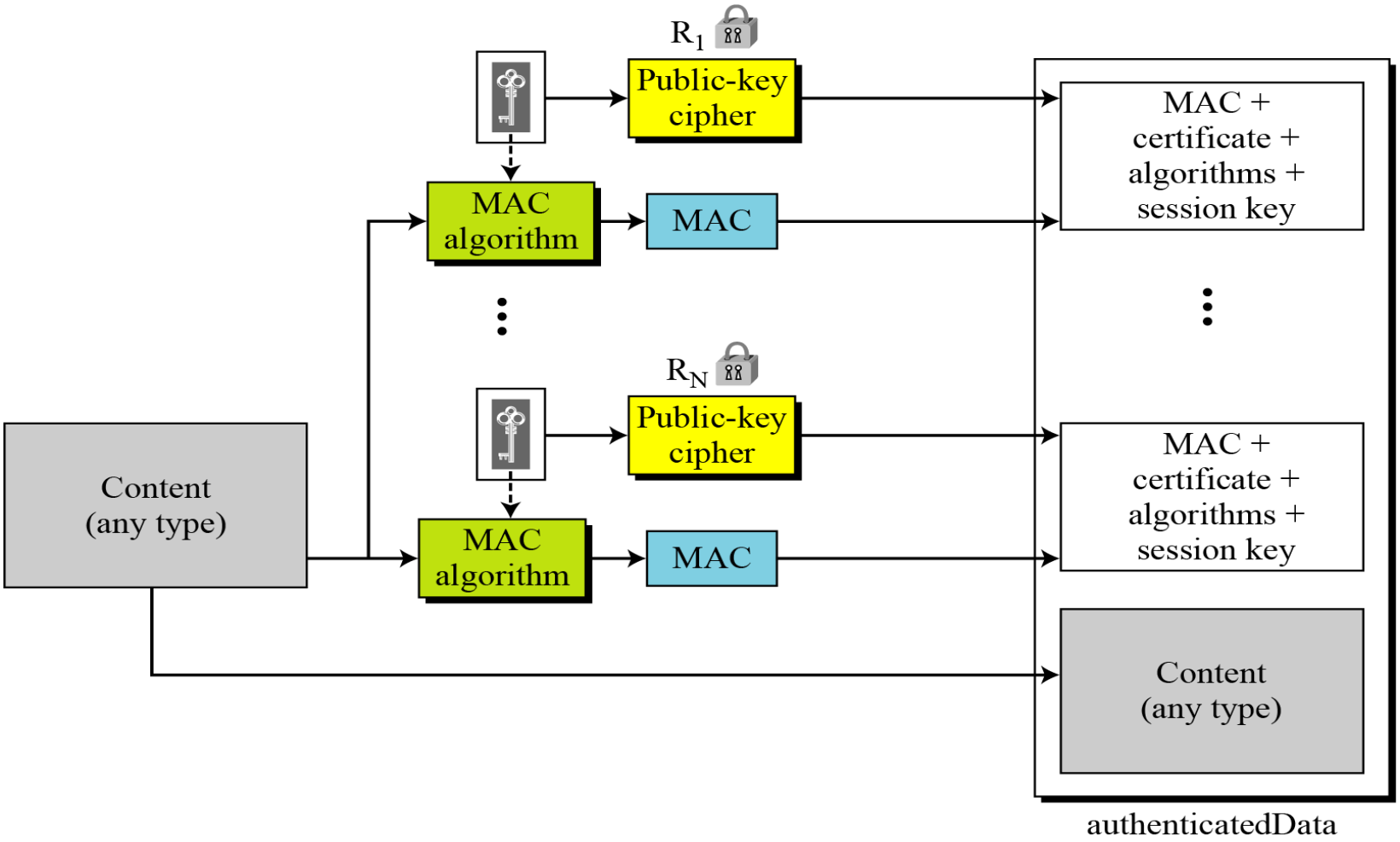


Figure 16.30 *Authenticated-data content type*

R_1 Encrypted with public key of recipient 1

R_N Encrypted with public key of recipient N



Cryptographic Algorithms

S/MIME defines several cryptographic algorithms. The term "must" means an absolute requirement; the term "should" means recommendation.

Table 16.17 *Cryptographic algorithm for S/MIME*

Algorithm	*Sender must support*	*Receiver must support*	*Sender should support*	*Receiver should support*
Content-encryption algorithm	Triple DES	Triple DES		1. AES 2. RC2/40
Session-key encryption algorithm	RSA	RSA	Diffie-Hellman	Diffie-Hellman
Hash algorithm	SHA-1	SHA-1		MD5
Digest-encryption algorithm	DSS	DSS	RSA	RSA
Message-authentication algorithm		HMAC with SHA-1		

16.3.2 Continued

Example 16.3

The following shows an example of an enveloped-data in which a small message is encrypted using triple DES.

Content-Type: application/pkcs7-mime; mime-type=enveloped-data
Content-Transfer-Encoding: Radix-64
Content-Description: attachment
name="report.txt";
cb32ut67f4bhijHU21oi87eryb0287hmnklsgFDoY8bc659GhIGfH6543mhjkdsaH23YjBnmN
ybmlkzjhgfdyhGe23Kjk34XiuD678Es16se09jy76jHuytTMDcbnmlkjgfFdiuyu678543m0n3h
G34un12P2454Hoi87e2ryb0H2MjN6KuyrlsgFDoY897fk923jljk1301XiuD6gh78EsUyT23y

PKCS Standards Summary

PKCS Standards Summary			
	Version	Name	Comments
PKCS #1	2.1	RSA Cryptography Standard[1]	See RFC 3447. Defines the mathematical properties and format of RSA public and private keys (ASN.1-encoded in clear-text), and the basic algorithms and encoding/padding schemes for performing RSA encryption, decryption, and producing and verifying signatures.
PKCS #2	-	*Withdrawn*	No longer active as of 2010[update]. Covered RSA encryption of message digests; subsequently merged into PKCS #1.
PKCS #3	1.4	Diffie–Hellman Key Agreement Standard[2]	A cryptographic protocol that allows two parties that have no prior knowledge of each other to jointly establish a shared secret key over an insecure communications channel.
PKCS #4	-	*Withdrawn*	No longer active as of 2010[update]. Covered RSA key syntax; subsequently merged into PKCS #1.
PKCS #5	2.0	Password-based Encryption Standard[3]	See RFC 2898 and PBKDF2.
PKCS #6	1.5	Extended-Certificate Syntax Standard[4]	Defines extensions to the old v1 X.509 certificate specification. Obsoleted by v3 of the same.
PKCS #7	1.5	Cryptographic Message Syntax Standard[5]	See RFC 2315. Used to sign and/or encrypt messages under a PKI. Used also for certificate dissemination (for instance as a response to a PKCS#10 message). Formed the basis for S/MIME, which is as of 2010[update] based on RFC 5652, an updated Cryptographic Message Syntax Standard (CMS). Often used for single sign-on.
PKCS #8	1.2	Private-Key Information Syntax Standard[6]	See RFC 5208. Used to carry private certificate keypairs (encrypted or unencrypted).
PKCS #9	2.0	Selected Attribute Types[7]	See RFC 2985. Defines selected attribute types for use in PKCS #6 extended certificates, PKCS #7 digitally signed messages, PKCS #8 private-key information, and PKCS #10 certificate-signing requests.
PKCS	1.7	Certification Request	See RFC 2986. Format of messages sent to a certification authority to

PKCS Standards Summary(Contd…)

PKCS #11	2.20	Cryptographic Token Interface[9]	Also known as "Cryptoki". An API defining a generic interface to cryptographic tokens (see also Hardware Security Module). Often used in single sign-on, Public-key cryptography and disk encryption[10] systems.
PKCS #12	1.0	Personal Information Exchange Syntax Standard[11]	Defines a file format commonly used to store private keys with accompanying public key certificates, protected with a password-based symmetric key. PFX is a predecessor to PKCS#12.This container format can contain multiple embedded objects, such as multiple certificates. Usually protected/encrypted with a password. Usable as a format for the Java key store and to establish client authentication certificates in Mozilla Firefox. Usable by Apache Tomcat, but not by Apache HTTP Server.
PKCS #13	–	Elliptic Curve Cryptography Standard[12]	*(Under development as of 2011[update].)*[13]
PKCS #14	–	Pseudo-random Number Generation	*(Under development as of 2011[update].)*[13]
PKCS #15	1.1	Cryptographic Token Information Format Standard[14]	Defines a standard allowing users of cryptographic tokens to identify themselves to applications, independent of the application's Cryptoki implementation (PKCS #11) or other API. RSA has relinquished IC-card-related parts of this standard to ISO/IEC 7816-15.[15]